脆弱性検知条件の供給源の再検討 —検知精度によるNVD・VulnCheck・一次データソースの比較とLLMによる構造化—

中岡 典弘[1,a)]　篠原 俊一[1,b)]

概要：我々はOSS脆弱性スキャナVulsと脆弱性DBを運用している．CVE急増を受け，検知に不可欠な「どの製品のどの版が影響を受けるか」の供給源を調べた．NVD・VulnCheck NVD++の全件（取下げ除き353,704件）では，NVDのCPE付与率が2023年以前の100%から2026年56.1%へ低下し，2025年分の37.3%はNVD自身が解析対象外と明示していた．ただし付与率は検知精度を予測しない．データソースのみを変えたDBで10環境を検知すると，NVDのみの未検知率はWindows Server 2022の1.3%からサポート終了版FortiOSの76.4%まで開き，その93.8%は組込みOSS部品であった．macOSでは下限のないCPEにより過剰検知が81.5%に達した．VulnCheckはMicrosoft Edgeの未検知を96.2%から0.4%へ下げる一方，Windowsでは過剰検知率70.6%を生み，独自CPEは日単位で入れ替わる．一次データソースが常に優るとは限らず，供給源の選択は製品ごとに異なる．本稿はその判断に要る測定と，一次データソースのLLM構造化方式を示す．

キーワード：脆弱性検知，脆弱性データベース，NVD，一次データソース，LLM

Revisiting the Supply of Vulnerability Detection Conditions: Comparing NVD, VulnCheck and Primary Sources by Detection Accuracy, with LLM-assisted Structuring

Norihiro Nakaoka[1,a)]　Shunichi Shinohara[1,b)]

Abstract: We develop Vuls, an open-source vulnerability scanner, and its database. Given the CVE surge, we ask where a scanner should get its detection conditions. Across all 353,704 non-rejected entries, NVD's CPE coverage fell from 100% through 2023 to 56.1% in 2026, with 37.3% of 2025 entries marked Deferred. Coverage, however, does not predict accuracy. Across ten environments differing only in input source, NVD's miss rate ranges from 1.3% on Windows Server 2022 to 76.4% on an end-of-life FortiOS, 93.8% of the latter embedded OSS components. How it errs differs too: on macOS, CPEs unbounded below drive an 81.5% false-positive rate. VulnCheck cuts Microsoft Edge's miss rate from 96.2% to 0.4%, yields 70.6% false positives on Windows, and rewrites its own CPEs daily. Nor is the primary source always better; the choice differs by product. We report the measurements that decision needs and a reproducible LLM-assisted method for structuring primary sources.

Keywords: Vulnerability Detection, Vulnerability Database, NVD, Primary Data Source, Large Language Model

1. はじめに

我々はオープンソース脆弱性スキャナ Vuls[1] と，そ

[1] フューチャー株式会社
Future Corporation
[a)] n.nakaoka.46@future.co.jp
[b)] s.shinohara.yx@future.co.jp

の脆弱性データベース群を継続的に開発・運用している．前稿 [2] では，一次データソース（ベンダ等が自ら公開するアドバイザリ）を可能な限り直接収集し，取得したままの `vuls-data-raw` 形式と共通スキーマの `vuls-data-extracted` 形式に分けて Git で履歴管理する「データハーベスト」方式を提案した [3], [4]．本稿はその基盤の上で，「検知条件を誰から得るべきか」という問いを定量的に扱う．

脆弱性スキャナが判定に用いる情報を本稿では**検知条件**と呼ぶ．検知条件とは最低限「どの製品の」「どのバージョン範囲が」影響を受けるかの組であり，これが機械可読な形で与えられなければスキャナはインベントリと突き合わせられない．NVD[5] においてこの役割を担うのが，`configurations` に格納される CPE 適用条件である．各条件は `criteria` に CPE 2.3 形式の文字列を持ち，任意で `versionEndExcluding` 等の範囲指定を伴う．ただしバージョン部が `*` や `-` で範囲指定を持たない場合，その条件は製品全体を影響対象とみなすことしか表現できない．

近年，CVE の報告件数は急増している．我々の観測では 2026 年は 7 月 29 日までの 210 日間で 44,809 件（取下げを除く）が公開され，2023 年の 28,817 件の約 2.7 倍のペースである．この増加が解析体制を圧迫していることは，NVD が `Deferred` というステータスの運用を始めたこと [6] に表れている．これは遅延ではなく NVD がエンリッチメントを行わないと明示した状態である．またこの遅延を受け，VulnCheck は独自の解析を加えた VulnCheck NVD++[7] の提供を開始した．NVD 由来の `configurations` を保持したまま，独自の結果を `vcConfigurations` と `vcVulnerableCPEs` に格納するため，両者の寄与を機械的に切り分けられる．

検知ツールの提供者にとってこれは，「これまで信頼して使えていたデータソースを今後も同じ前提で使えるのか」という運用上の問題である．しかし我々の知る限り，CVE 増加後の NVD と VulnCheck について，検知条件の供給という観点から定量的に整理した報告はない．

供給側に頼れない製品では一次データソースを直接収集することになるが，HTML はスキーマを持たず提供元の改修でセレクタが予告なく変わり，記述も多様で値が欠けている．LLM はこの両者に適するが，日々のデータ経路に置くと前稿で確立した再現性を損なう．

そこで本稿は，NVD と VulnCheck NVD++の全件（2026 年 7 月 29 日時点，取下げを除く 353,704 件）について CPE 適用条件の付与率・粒度・付与遅延・その後の安定性を測り，さらに一次データソースと突き合わせる 4 つの観点を定義して Fortinet と Cisco で実測する（3 章）．そのうえで，一次データソースを LLM で構造化する 3 つの適用パターンを整理して実装し（4 章），入力データソースのみを変えた 4 種のデータベースで計 10 環境を検知して，上記の差が検知結果に現れることを確認する（5 章）．NVD のみでの未検知率は Windows Server 2022 の 1.3%からサポート終了版 FortiOS の 76.4%まで開き，3 章で測った付与率はこの差をほとんど予測しない．

2. 関連研究

Dong ら [8] は，NVD のエントリと CVE 概要文・参照先の非構造化レポートとの不整合を検出するシステム VIEM を提案し，約 20 年分・78,296 件の CVE ID を対象とした調査で，NVD エントリと影響バージョンが完全に一致するのは 59.82%にすぎないことを示した．

本研究は Dong らと問題意識を共有するが，VIEM が NVD にエントリが存在することを前提に不整合を測るのに対し，本稿は CVE 増加後に顕在化した「検知条件がそもそも生成されない」という失敗モード（`Deferred`）を対象とし，検知精度への影響まで評価する点で異なる．また抽出精度単体ではなく継続運用における保守性・再現性を設計目標に置く．

NVD のデータ品質については，Anwar ら [9] が公開日・ベンダ名・深刻度等の不整合と欠損を体系的に調査し，Nguyen と Massacci[10] が Google Chrome を対象に影響バージョンデータの系統的な誤りを示している．いずれも CVE 増加以前の調査であり，本稿は同種の問題が 2024 年以降に「欠損」として急拡大したことを示す点で相補的である．

3. 検知条件の供給実態の定量調査

本章は検知条件の供給を，NVD・VulnCheck 内部の情報だけによる全件調査（3.2〜3.3 節）と，一次データソースとの比較（3.4〜3.5 節）の 2 つから測る．

3.1 調査対象と方法

全件調査の対象は，我々のデータ収集基盤が取得した NVD JSON 2.0 データフィード [11]（371,416 件）と VulnCheck NVD++ (`nist-nvd2`) API[7]（371,393 件）の，2026 年 7 月 29 日時点のスナップショットである．両者の CVE ID 集合を比べると，NVD のみに存在するものが 23 件，VulnCheck のみに存在するものが 0 件で，VulnCheck NVD++ の収録 ID は NVD の ID 空間を超えない．集計では `vulnStatus` が `Rejected` の 17,712 件を除外し 353,704 件を対象とし，各エントリの `configurations`（NVD）と `vcConfigurations`（VulnCheck 独自）から `vulnerable: true` の `cpeMatch` の個数とバージョン指定の種別を数えた[*1]．

[*1] 取下げエントリは製品情報を持たないことが正常であり，含めると付与率を不当に低く見せる．実例として，Fortinet 採番の CVE には 2025 年に一括取下げされたものが 866 件含まれ，除外しない場合 Fortinet の付与率は 32.8%，除外後は 99.6%と算出される．

一次データソースとの比較には次の 4 基準を定める．

C1（エントリ欠落）一次データソースに存在する脆弱性が NVD・VulnCheck にエントリを持たない．両者は CVE ID を主キーとするため，CVE ID 未採番の脆弱性は構造的に収録され得ない（**C1a**）．上記のとおり VulnCheck は C1 を改善しない．

C2（製品欠落）一次データソースが affected とする製品が相手側に列挙されていない．

C3（バージョン不一致）affected または fixed の範囲が異なる．相手側が狭ければ未検知（C3a），広ければ過剰検知（C3b）を生む．

C4（付与遅延）一次データソースでの公開日から検知条件が付与されるまでの日数．

一次データソースには，Fortinet の PSIRT Advisory（CVRF および CSAF[12]）と Cisco の openVuln API 由来の JSON[13] を用いる．CVRF の `product_statuses` は `Known Affected` しか持たず，影響を受けない版は「`Please upgrade to FortiADC version 7.2.1 or above`」のように `Solutions` 注記の自然文としてしか書かれない．そこでこの注記を解析して負例を作った（122 件から 511 組，影響版と矛盾するものは 0 件）．CSAF は影響を受けない版を `known_not_affected` で機械可読に与える（Fortinet は `fixed` を用いない）．Fortinet は 2024 年に注記の記入をやめ CSAF の公開を始めており，両者は期間を分担する．製品情報を伴う CVRF は 2022 年以降の 616 件，CSAF は 173 件である．Cisco は `productNames` に「Cisco IOS XE Software 16.12.1」のような自然文で製品を列挙し，CPE を持たないため，製品名から CPE への対応付けを別途実装した．しかし，対応付けられるのはアドバイザリ単位では 31.9%にとどまる．そこで Cisco は C1 と C4 のみ測定し，C2・C3 に相当する比較は 5 章の検知結果として示す．

C2・C3 の判定には**境界値法**を用いる．連続区間として比較すると，Fortinet の `<=6.4.15` と NVD の `versionEndExcluding: 6.4.16` のように，実在する版の集合としては同一のものが「NVD 側が広い」と誤判定されるためである．そこで (CVE, 製品) ごとに影響ありとする版を検知されるべき点，影響なしとする版を検知されるべきでない点とし，各点で相手側の条件の真偽が一致するかを評価する．製品の突合は CPE の製品部から区切り文字を除いた文字列で行い，版表現が解析できない組は除外した．さらに結果を CVE の採番元（NVD の `sourceIdentifier`）で層別する．Fortinet 自身が採番した CVE と，Fortinet 製品に組み込まれた部品に対して他者が採番した CVE（以下，**組込み部品の CVE**）では NVD の振る舞いが大きく異なるためである．

C4 は Git 履歴から測る．スナップショットを 1 日 2 回取得し Git で履歴管理しており [4]，履歴は NVD が 2025 年 6 月 26 日，VulnCheck が 2025 年 8 月 6 日以降利用できる．各 CVE のファイルを辿り，`vulnerable` な `cpeMatch` が初めて現れたコミットの日付を求めた．

3.2 CPE 適用条件の付与率の推移

公開年別の解析状況を集計した．対象は 2020〜2023 年の 92,362 件，2024 年 39,957 件，2025 年 48,162 件，2026 年 44,809 件（1 月 1 日〜7 月 29 日）である．

(1) 2023 年公開分までは付与率 100%である．

2016 年から 2023 年までのいずれの年も，取下げを除く全エントリに CPE 適用条件が付与されている．検知条件の有無という観点では，CVE 増加前の NVD は全件を網羅している．記述内容が正しいかは別問題であり [8], [10]，5 章で評価する．

(2) 2024 年を境に急落し，待っても解消しない．

`configurations` に `vulnerable` な `cpeMatch` を 1 つ以上持つ割合は 2024 年 80.5%, 2025 年 62.6%, 2026 年 56.1%と低下し，低下分はほぼそのまま `Deferred` の増加（19.5%，37.3%，34.8%）で説明される（2026 年は別途 9.3%が解析待ちである）．未付与エントリの滞留も長く，`Awaiting Analysis` で中位数 43 日，`Deferred` の 41,363 件は中位数 286 日が経過している．2025 年以降の NVD は 4 割弱が将来にわたって供給されず，新規公開の CVE も公開後 1〜4 か月は検知条件が存在しない前提で設計する必要がある．

(3) VulnCheck は補完するが埋め切らない．

`vcConfigurations` を加えると付与率は 2024 年 88.3%，2025 年 76.1%，2026 年 78.5%へ上がり，NVD が付与せず VulnCheck のみが付与したエントリは 3,083 件・6,492 件・10,058 件（同年の 7.7%・13.5%・22.4%）である．それでも 2025 年以降は 2 割強が両者ともに未付与である．

3.3 付与された条件の粒度と CNA 別の偏り

CPE 適用条件が存在することとバージョンを特定できることは同じではない．2024 年以降に公開された 132,928 件のうち，バージョン範囲または特定バージョンで表現されているのは NVD で 63.9%，VulnCheck を加えて 77.8%にとどまる．

採番 CNA 別（`sourceIdentifier` による）の差は極端である．Microsoft 99.7%，Fortinet 99.6%，Apple 99.4%と付与率がほぼ維持される一方，Cisco 77.2%（`Deferred` 15.0%），JPCERT/CC 採番は 39.1%（同 59.7%）で VulnCheck を加えても 48.4%にとどまる．JPCERT/CC は JVN を通じて国内ベンダの脆弱性を採番する経路であり，国内ベンダ製品の検知条件はほぼ供給されていない．また Microsoft は付与率 99.7%に対しバージョン特定可能率が 92.9%と乖離する．版を特定できないのは Office や Microsoft 365 のように CPE に版を持たない製品で，Windows の CPE は 99.3%がビルド番号の範囲を伴う．

3.4 一次データソースとの一致（C1〜C3）

まず C1 を見る．Fortinet の 1,119 件のアドバイザリのうち 37 件，Cisco の 5,309 件のうち 260 件（4.9%）が CVE ID を持たない（いずれも古い年に集中）．CVE ID を持つもののうち NVD に存在しないものは Fortinet で 0 件，Cisco で 8 件である．CVE ID 未採番のもの（C1a）は CVE 制度の設計に起因するため，一次データソースを直接収集する以外に解決手段がない．

C2・C3 の結果を表 1 に示す．対象は，CVRF が Fortinet 採番 656 CVE・1,135 組と他者採番 54 CVE・255 組，CSAF が同 168 CVE・298 組と 6 CVE・60 組である．

(1) 組込み部品の脆弱性では，NVD も VulnCheck も Fortinet 製品を 1 件も列挙しない．

他者が採番した CVE について C2 は 100.0%である（CVRF で 54 CVE・255 組，CSAF で 6 CVE・60 組）．そのほぼ全て（CVRF で 252 組，CSAF で 60 組）は CPE は付与されているが Fortinet 製品の CPE が 1 件も含まれないケースで，CVE-2024-6387（regreSSHion）や CVE-2023-48795（Terrapin）が該当する．Fortinet は自社製品が影響を受けることをアドバイザリで公表しているが，NVD の CPE には部品そのものの製品（`openbsd:openssh` 等）しか並ばない．OSS 部品を組み込んだ製品を検知しようとすると，この欠落がそのまま未検知になる．3.2 節の付与率にはこの欠落は一切現れない．VulnCheck の独自エンリッチメント（`vcConfigurations`）も同じく 100.0%であり，この欠落は埋まらない．4.2 節の補完表を適用すると 2021 年以前も測定でき，他者採番の CVE は 54 件・255 組から 241 件・1,084 組に増えるが，C2 は 95.4%と高いままである．

(2) Fortinet が採番した CVE でも一致は 63.1%にとどまる．

3.3 節で Fortinet の付与率は 99.6%と最良の水準であったが，内容の一致はそこまで高くない．C2（18.7%）の内訳は 212 組中 210 組が「NVD に Fortinet 製品の CPE はあるが，当該製品の CPE が無い」ケースで，製品の切り方の違いも含む．FortiClient Windows/Mac/Linux を Fortinet は別製品として数えるのに対し，NVD は `target_sw` で区別するため一致しない（うち 46 組）．付与率は検知可能性の上限にすぎない．一方 VulnCheck は CSAF 基準・Fortinet 採番で一致を 71.1%から 89.9%へ，C2 を 15.8%から 0.0%へ，未検知率を 10.8%から 1.5%へ改善する．過剰検知は CSAF 基準で負例 1,075 点中 NVD・VulnCheck とも 12 点（1.1%），CVRF 基準で負例 400 点中 NVD 4 点・VulnCheck 7 点であり，本測定の範囲では「VulnCheck が過剰検知を増やす」傾向は認められなかった（粗い CPE の影響は 5 章で改めて測る）．

3.5 付与の遅延と安定性

公開から検知条件が初めて現れるまでの日数を Git 履歴から測った[*2]．

全体では，NVD は中位数 7 日を要し 42.6%が執筆時点でも未付与であるのに対し，VulnCheck は中位数 1 日で未付与は 22.6%にとどまる．VulnCheck の価値は網羅性ではなく初動の速さにある．

ベンダ間の差も大きい．アドバイザリ公開日を起点とすると，Fortinet は NVD でも中位数 3 日・未付与 1.1%だが，Cisco は中位数 16 日で 48.3%が未付与である（VulnCheck でも 10.4%が残る）．付与率の低いベンダでは欠落だけでなく遅延も深刻である．

同じ Git 履歴から，付与された条件が維持されるかも見た．NVD の `configurations` は標本 320 件のいずれでも `part:vendor:product` の集合が変わらないのに対し，VulnCheck の独自フィールドでは製品そのものが出入りする．CVE-2015-4000 の `suse:suse_linux_enterprise_server` は，更新のあった 334 回のスナップショットのうち 305 回で有無が入れ替わる．3 回以上入れ替わる CVE は推定約 1 万件（2.9%）に及ぶ．昨日の検知結果が翌日には消え，差分が新たな脆弱性か供給側の揺れかを区別できない．

4. 一次データソースの構造化

4.1 課題と設計要件

1 章で述べた構造の不安定さと値の欠落に LLM は適するが，非決定性をデータ経路に持ち込むことはできない．そこで次の 4 要件を課す．

R1（決定性）同一入力から常に同一出力が得られる．

R2（検証可能性）出力を機械的に検証でき，通らないものは DB に入らない．

R3（保守コストの局所化）構造変更への追従が全面的な書き換えを要しない．

R4（監査可能性）生成元を追跡できる．

共通する設計は LLM の非決定性を日々のデータ経路から追い出すことであり，LLM に何を出力させるかで **P1**（データを一度だけ生成）・**P2**（抽出器のコードを生成）・**P3**（入力ごとに生成）の 3 つに整理した．この順に，LLM が日々のデータ経路に残る度合いが高くなる．LLM は機械可読形式の代替ではなく，それが無いか不完全な場合の補完手段である．

4.2 P1：欠損データの生成

構造はあるが値が欠けている場合，LLM にデータを出力させ，凍結資産としてコミットする．実行は一度きり

[*2] 全体は Git 履歴開始以降の CVE から 25 件ごとに抽出した標本（NVD 2,773 件，VulnCheck 2,549 件），Fortinet と Cisco は全件である．

表 1 Fortinet 一次データソースとの一致状況（境界値法による，単位%．各セルは NVD / VulnCheck）

Table 1 Agreement with the Fortinet primary source (boundary-value method; each cell is NVD / VulnCheck).

基準	採番元	一致	C2 製品欠落	C3a 未検知	C3b 過剰検知	未検知率	過剰検知率
CVRF	Fortinet	63.1 / 74.5	18.7 / 11.3	17.9 / 13.4	0.2 / 0.4	16.9 / 9.5	0.0 / 0.0
CVRF	他者	0.0 / 0.0	100.0 / 100.0	0.0 / 0.0	0.0 / 0.0	99.7 / 99.7	0.0 / 0.0
CSAF	Fortinet	71.1 / 89.9	15.8 / 0.0	9.4 / 6.4	3.7 / 3.7	10.8 / 1.5	0.5 / 0.5
CSAF	他者	0.0 / 0.0	100.0 / 100.0	0.0 / 0.0	0.0 / 0.0	59.2 / 59.2	0.0 / 0.0

一致・C2・C3a・C3b は (CVE, 製品) 組に対する割合，未検知率・過剰検知率は検証点に対する割合．負例は CVRF が `Solutions` 注記の 400 点，CSAF が `known_not_affected` の 1,075 点．CVRF は製品情報を伴う 2022 年以降が対象．

で日々の fetch では呼ばれない（R1）．Fortinet CVRF は 2021 年以前を中心に `product_statuses` が空で，1,119 件のうち 456 件が該当し，値は注記の自然文にしかない．CSAF は 2024 年以降しか存在せず，この期間は一次データソースから検知できない．そこで Fortinet 自身が書いた 4 つの記述——アドバイザリの 2 種の注記，CNA として公開する CVE レコード，旧来の手書きデータセット——を LLM に突き合わせさせ，補完表を生成した [14]．検証は出典間の相互照合による（R2）．バージョン単位の照合と注記・NVD との突合により計 118 件の不一致を検出し，根拠を記録しながら個別に裁定した（R4）．バージョン制約を持たない 12 行は製品全体に一致してしまうため個別に監査し，未監査の行や未知の製品名は抽出を失敗させる——黙って落とすことも黙って広げることもしない．それでも誤りは残り，のちにアドバイザリ本文を典拠に全行を作り直し，系統をまたいで広がる範囲 29 行などを修正した．結果として 456 件中 421 件が検知条件を得た．

4.3 P2：抽出器の生成

構造が安定していれば，fetch と extract のコードを LLM に書かせ，日々の収集は生成済みコードのみで決定的に動作させられる（R1）．検証はゴールデンセットへの回帰テストによる（R2）．5 章で用いる Apple Security Releases[15] と Microsoft の更新プログラム記事 [16] がこれにあたる．このパターンの作業量は，構造の解析ではなく例外の発見と確定が占める．Apple では 2003 年以前のアーカイブ形式や公開が停止された記事の扱いが大半で，合計 1,327 行を要した．このような網羅的な走査は LLM が得意とする作業であり，人手は生成された差分の承認に縮小される（R3）．CI は取得件数の対前回比を閾値で監視して DB への反映を止める．変更を再び LLM に与えて抽出器を直す運用は設計にとどまり，現状は人手で修正している．

4.4 P3：HTML からの直接変換

構造が弱く鍵も欠けている場合，新規入力ごとに LLM にデータを出力させる．ここだけはデータ経路に LLM が残るため，変換結果を Git で持つことで決定性を作り（R1），型と検証器で検査する（R2）．本稿が調べた範囲で最も極端なのは OpenSSH であった．第一に，項目の境界が位置にしか現れない．セキュリティ情報のページ [17] の 55 件のエントリは `<li>` の中で日付・影響範囲・表題・本文が `<br>` で区切られているだけで `class` 属性も見出しも持たず，`<br>` の数は 2 個から 7 個までばらつくため，何番目の断片が影響範囲かをセレクタで固定できない．第二に，鍵となる CVE ID がほとんど振られていない．本文に CVE ID を含むエントリは 6 件にとどまる．OpenSSH は CNA ではなく，我々が観測した 134 件の CVE は MITRE や Red Hat が採番している．アドバイザリを書く主体と CVE レコードを書く主体が別であることが欠落の理由である．その帰結が 3.4 節の CVE-2024-6387（regreSSHion）である．ページはこれを `8.5p1 to 9.7p1 (inclusive)` と正確な範囲付きで記述しながら CVE ID を書いていない．検知条件は一次データソースだけで組み立てられるが，それが何と呼ばれている脆弱性なのかは分からない．

55 件のエントリをすべて `vuls-data-raw` 形式へ変換した [18]．検査は 2 段ある．extract のコードは，各バージョン文字列が OpenSSH のリリース表記であること，範囲の上下限がそれぞれ高々 1 つであり範囲が空でないことを検査し，通らなければ抽出を失敗させる．一方，出力の版と CVE ID が原文に逐語で現れるか，変換元の `<li>` を連結すると元のリストが復元されるかは，手順書が LLM に課す自己検査にとどまり，機械的には強制していない．原文にない値の混入を機械的に止められていない点で，R2 を満たしていない．CVE ID の紐付けは一次データソースだけでは 6 件（10.9%）にとどまるが，MITRE の CVE レコードを参照することで 45 件（81.8%）まで上がった．一次データソースは影響範囲を持ちながら索引を持たず，MITRE はその逆であるという相補性が確認できた．ただしこの紐付けは検知件数を 1 件も変えていない．生成元の追跡（R4）は，fetch がページを `origin/` に逐語で保存し，記録ごとに変換元の `<li>` を持たせることで満たす．変換の陳腐化——ページが変わったのに `raw/` を作り直していない状態——は CI で検出する [19]．ページが変わらない限り LLM は走らず，出力は取得し直しても変わらな

い（R1）．残る不確かさは，ページが変わったときの再変換が同じ出力になる保証がないことである．

5. 検知精度への影響の評価

3 章は一次データソースの全アドバイザリを走査して検知条件そのものを比較した．本章は製品とバージョンを 10 点選び，検知結果を比較する．

5.1 実験設定

同一のスキャナ [1] に，入力データソースのみを変えた 4 種のデータベースを与える．**DB-primary** は一次データソース（Fortinet CVRF/CSAF，Cisco openVuln API，Microsoft の MSRC CVRF・Security Bulletin・更新プログラムカタログ・WSUSSCN2・更新プログラム記事，Apple Security Releases，OpenSSH Security），**DB-nvd** は NVD の `configurations`，**DB-vc** は VulnCheck の独自エンリッチメント（`vcConfigurations` と `vcVulnerableCPEs`），**DB-nvd+vc** は両者である．DB-vc に NVD 由来の `configurations` は含めない．VulnCheck が独自に何を足し何を誤るかを分離するためである．なお Vuls は，検証済みソース（NVD 等）が当該 CVE にその製品の CPE を定義していれば，バージョンが一致せず非検知であっても補助的ソース（VulnCheck）の同一製品の検知を捨てる．このため DB-nvd+vc は DB-nvd と DB-vc の和集合にならない．FortiOS 7.0.1 で DB-vc の過剰検知 44 件が DB-nvd+vc で 0 件になるのはこの規則による．

同一のスキャン結果を 4 つ複製しデータベースだけを変えて検知した．対象は FortiOS 7.0.1 と 5.2.0，Cisco IOS 15.6(1)T1 と IOS XE 16.12.1，Windows 11 23H2 と Windows Server 2022，macOS 14.7.4 と 10.13.3（系統の最終版 10.13.6 の手前の版），OpenSSH 8.5p1 と 9.6p1 の 10 環境である．1 台に複数の製品が載るため（macOS と Safari 等），検知は照合された CPE により製品単位に集計した（Safari は本文で述べる）．Microsoft に限り，一次データソースによる検知には適用済み KB を用い，スキャン結果も実機から取得した（他は宣言した CPE のみを持つ疑似サーバである）．なお `apple:mac_os_x` は `apple:macos` に寄せた．NVD が古い CVE を前者の CPE で記述しており，分けたままでは表記ゆれが NVD の未検知として計上されるためである．

DB-primary が検知し DB-X が検知しない CVE を**未検知**，逆を**過剰検知**とする．

5.2 結果

結果を表 2 に示す．

(1) C2（製品欠落）はそのまま未検知になるが，VulnCheck が埋める場合もある．

FortiOS 5.2.0 では DB-nvd の未検知率が 76.4%（97/127 件）に達し，未検知 97 件の 93.8%（91 件）は Fortinet 以外が採番した CVE である．CVE-2014-3566（POODLE）のように，Fortinet が影響を公表している OSS 部品の脆弱性で，3.4 節で C2 が 100.0%であった欠落が長く運用される版ほど累積する（現行系の 7.0.1 では 10.7%）．Microsoft Edge では逆に DB-nvd の未検知率が 96.2%で，その 99.9%は Chrome が採番した CVE である．NVD は Chromium の脆弱性に `google:chrome` の CPE しか付けず，Edge を列挙しない．DB-vc はこれを 0.4%まで下げるが，3.4 節では Fortinet の組込み部品を埋めなかった．VulnCheck が組込み部品を埋めるかは製品によって一貫しない．

(2) NVD は Apple に下限を書かず，過剰検知が支配的になる．

DB-nvd の過剰検知率は macOS 14.7.4 で 38.5%（342 件），10.13.3 で 81.5%（1,658 件）に達する一方，未検知率は 6.7%・15.3%にとどまる．原因は範囲の書き方で，前者は全件，後者の 99.4%が，下限を持たない `cpeMatch`（`apple:macos:*` に `versionEndExcluding` のみ）により一致する．たとえば CVE-2025-43212 は macOS Sequoia 15.6 での修正だが，上限しか書かれないため 14.7.4 も一致する．この書き方は Apple の CPE の 14.0%（23,164 件中 3,237 件）にすぎないが，1 件が当該系統より古い全版に一致する．DB-vc の過剰検知率はさらに高く（49.8%・90.0%），10.13.3 の 2,572 件中 1,526 件は NVD が書いた範囲を VulnCheck が外したものである．CVE-1999-1076 では NVD の `macos:9` が `macos:-` となり，全 macOS に一致する．付与率 99.4%（3.3 節）は範囲の妥当性を保証しない．同じホスト上の Safari 18.3.1 では未検知 16.6%・過剰 34.9%と両方向に外す．これは Apple に限らない．FortiOS 5.2.0 への過剰検知 24 件も 23 件が下限を持たない範囲によるもので，Fortinet が系統内の上限として書く「5.4.4 and below」を NVD は旧系統まで広げる．

(3) VulnCheck の自動付与は，Windows で大量の過剰検知を生む．

Windows 11 23H2 に対し DB-vc は DB-primary と 1 件も一致せず，835 件を過剰検知した．原因は NVD の抽象 CPE の展開である．CVE-2007-1644 に NVD は `all_windows:abstract_cpe` という照合不能なプレースホルダを置くが，VulnCheck はこれを `windows_11_23h2:-` へ展開し，2007 年の Windows NT/2000 の DNS 脆弱性が Windows 11 で検知される．この 835 件のうち 833 件は DB-nvd+vc でも抑制されず，過剰検知率は 70.6%に達する．機械的な CPE 生成の帰結である [20]．

(4) Cisco では，NVD が未検知と過剰検知の両方向に外す．

Cisco だけは両方向の誤りが大きい．しかもこの未検知は組込み部品ではなく，IOS XE の未検知 80 件の 98.8%は Cisco 自身が採番した CVE である．原因は条件の書

表 2 データベースを変えたときの検知結果（DB-primary を基準とした未検知率／過剰検知率，単位%）

Table 2 Detection results by database, as miss rate / false-positive rate against the primary-source database.

スキャン対象（製品・バージョン）	基準件数	DB-nvd	DB-vc	DB-nvd+vc
FortiOS 7.0.1	169	10.7 / 0.0	14.8 / 23.4	10.1 / 0.0
FortiOS 5.2.0	127	76.4 / 44.4	74.8 / 50.0	75.6 / 43.6
Cisco IOS 15.6(1)T1	136	53.7 / 23.2	54.4 / 85.6	46.3 / 36.5
Cisco IOS XE 16.12.1	214	37.4 / 14.1	30.4 / 40.9	28.0 / 20.6
Windows 11 23H2	357	2.8 / 0.6	100.0 / 100.0	2.8 / 70.6
Windows Server 2022	676	1.3 / 4.9	1.0 / 0.6	1.3 / 4.9
Microsoft Edge 146	1,862	96.2 / 1.4	0.4 / 1.4	0.5 / 0.2
macOS 14.7.4	586	6.7 / 38.5	4.8 / 49.8	6.5 / 42.0
macOS 10.13.3	445	15.3 / 81.5	35.7 / 90.0	12.1 / 81.5
OpenSSH 8.5p1	7	14.3 / —	57.1 / —	14.3 / —
OpenSSH 9.6p1	5	20.0 / —	60.0 / —	0.0 / —

未検知率は基準件数に対する割合，過剰検知率は当該 DB の検知件数に対する割合．Microsoft Edge は 2 台の Windows に同一バージョンが導入されているため 1 行にまとめた．OpenSSH の過剰検知率は定義できない（5.2 節 (6)）．

き方にある．NVD の Cisco の CPE は版の列挙が主体で，`cisco:ios` の 67,803 件の `cpeMatch` のうち範囲指定は 131 件にすぎず，列挙から漏れた版はそのまま未検知になる．過剰検知側は NVD が余分に版を挙げた場合と Cisco 側が版を書いていない場合を含み，後者は CPE に対応付けられる 1,703 件のうち 373 件（21.9%）に及ぶ．

(5) 供給源の信頼度に基づく抑制は，正しさとは相関しない．

5.1 節の抑制規則は，製品によって正反対に働く．Cisco IOS XE では抑制された 112 件のうち 31 件（27.7%）を DB-primary も検知している（＝真陽性を捨てている）．一方 macOS 10.13.3 では 1,501 件を抑制して取りこぼしは 5 件にとどまり，Windows 11 23H2 では抑制されずに通った 833 件のうち DB-primary と一致するものは 0 件であった．抑制の可否は NVD が製品を定義しているかだけで決まり，正しさとは独立だからである．その代償と限界は 5.4 節で述べる．

(6) OpenSSH では，一次データソースを正解とみなす前提そのものが崩れる．

OpenSSH だけは一次データソースの検知件数がどの DB よりも少ない．8.5p1 では DB-primary 7 件に対し DB-nvd 24 件, DB-vc 94 件（9.6p1 では 5 件に対し 19 件, 95 件）である．ただし DB-nvd との差は過剰検知と呼べない．DB-nvd のみが検知した 18 件・15 件は一次データソースに現れず，すべて MITRE 採番で，8.5p1 では 67%，9.6p1 では 80%が 2026 年公開分である．OpenSSH 自身は AI による報告の多くを影響なしと判断しており [21]，何を脆弱性と数えるかで上流と第三者採番者の判断が割れるためである．どちらを正解とするかは事前に決められない．一方 DB-vc だけが検知した 73 件の大半は (2) と同じ `openbsd:openssh:-` で，1999 年の脆弱性が 8.5p1 で検知される（DB-nvd+vc で 71 件が抑制されるのは 5.1 節の規則が正しく働いた例である）．未検知の内訳も異質である．DB-nvd の取りこぼしは各 1 件で，8.5p1 のそれは CVE 未採番で NVD には原理的に存在し得ない（3.4 節の C1a）．DB-vc は 8.5p1 で 4 件，9.6p1 で 3 件を取りこぼし，regreSSHion と Terrapin を含む．原因はベンダ名の不一致（VulnCheck は `openssh:openssh`，他は `openbsd:openssh`）で，同じ製品を指す CPE が供給源ごとに違うだけで，最も広く知られた脆弱性が検知されない．

5.3 どの時点までの NVD を信頼してよいか

3.2 節では 2023 年公開分までの NVD の CPE 付与率は 100%であった．しかし検知結果はそれを裏切る．FortiOS 5.2.0 の未検知 97 件（76.4%）はすべて 2023 年以前の CVE であり，Cisco IOS でも 2023 年以前の未検知率は 56.0%である．macOS 10.13.3 は基準 445 件がすべて 2023 年以前だが，そこでの誤りは未検知（15.3%）ではなく過剰検知（81.5%）として現れる．古い年ほど条件が整うのではなく誤り方が変わるだけで，「CVE 増加前の NVD なら信頼してよい」という仮説は成り立たない．付与率の低下は 2024 年以降の問題だが，ここでの誤りは以前から付与率に現れない形で存在する．逆に Cisco では新しい年ほど悪く，IOS XE の未検知率は 2023 年以前 34.7%に対し 2026 年は 8 件中 8 件である．DB-vc はこれを 0%まで下げており，3.5 節の Cisco の付与遅延（NVD 中位数 16 日，VulnCheck 1 日）が検知結果に現れている．

5.4 限界と考察

一次データソースを正解とみなす前提の限界．

本章の未検知率・過剰検知率は DB-primary を正解とみなした値だが，一次データソース側にも欠落がある（5.2 節

(4)(6))．macOS 10.13.3 の過剰検知 1,658 件のうち 425 件は Apple のアドバイザリに現れず，Apple 側の欠落の可能性を排除できない．また 5.1 節のとおり Microsoft のみ検知方式が異なる．結果はバージョンの選び方にも依存する（同じ FortiOS で未検知率 10.7%と 76.4%）．指標にも制約があり，表 1 の C2 には 3.4 節の突合の失敗が含まれ，「自社採番か組込み部品か」の判定は NVD の `sourceIdentifier` を代理指標としている．

供給源の信頼度による抑制の代償と限界．

VulnCheck の CPE は機械的に生成されるため，うまく機能するものと大量の偽陽性を生むものが混在する（5.2 節 (1)(3)(6)）．Vuls の抑制規則はこれに対する妥協である．偽陽性 833 件がすり抜けるのは，NVD の抽象 CPE（`all_windows`）と VulnCheck が展開した具体 CPE（`windows_11_23h2`）が別製品として扱われるためで，規則を緩めれば真陽性 31 件は救えるが偽陽性は悪化する．必要なのは信頼度の調整ではなく，照合不能な抽象 CPE をソース側で明示的に扱うことである．

LLM 利用に伴うリスク．

4 章の P1・P2 は非決定性を生成時に閉じ込めるが，P3 はデータ経路に LLM が残る．4.4 節のとおり原文にない値の混入を止める検査を機械的に強制できておらず，4.2 節では 118 件の不一致を裁定してなお誤りが残っており，単一の出力を信じてはならない．

6. おわりに

本稿では，脆弱性検知に不可欠な検知条件が現在どこから供給されているかを定量的に調査した．NVD の CPE 付与率は 2023 年までの 100%から 2026 年には 56.1%へ低下し，低下分の大半は NVD が解析対象外とした `Deferred` に対応する．しかし付与率は検知精度を予測しない．組込み部品の脆弱性について NVD も VulnCheck も Fortinet 製品の CPE を 1 件も列挙せず，Fortinet が採番した CVE でも一致は 63.1%にとどまった．入力データソースのみを変えた 4 種のデータベースで 10 環境を検知すると，NVD のみでの未検知率は製品によって 1.3%から 76.4%まで開き，VulnCheck は付与率を上げる一方で Windows に 70.6%の過剰検知を生んだ．誤り方も製品で違い，Cisco では未検知 53.7%と過剰検知 23.2%が同時に，Apple では下限を持たない CPE による過剰検知 81.5%が生じた．誤りは 2023 年以前の CVE でも減らず，「CVE 増加前の NVD なら信頼してよい」とはいえない．併せて，我々自身が置いている抑制規則の代償（Cisco IOS XE では抑制 112 件中 31 件が真陽性）と限界（偽陽性 833 件のすり抜け）を定量化した．

一次データソースの構造化を，欠損データの生成・抽出器の生成・HTML からの直接変換の 3 パターンに整理し，Fortinet・Apple・Microsoft・OpenSSH に実装した．今後は，一次データソースから取得すべき製品と NVD や VulnCheck で十分な製品を，より広い範囲で見分ける必要がある．製品ごとに取得元を判断できなければ，収集コストは正当化できない．

参考文献

[1] Vuls: VULnerability Scanner. https://github.com/future-architect/vuls.

[2] 中岡典弘, 篠原俊一. 広範な脆弱性情報の統合管理と履歴追跡. コンピュータセキュリティシンポジウム 2025 (CSS2025) 論文集, 2025.

[3] vuls-data-update. https://github.com/MaineK00n/vuls-data-update.

[4] vuls-data-db. https://github.com/vulsio/vuls-data-db.

[5] National Vulnerability Database. https://nvd.nist.gov/.

[6] Vulnerability Status. https://nvd.nist.gov/vuln/vulnerability-status.

[7] VulnCheck NVD++. https://docs.vulncheck.com/community/nist-nvd/attribution.

[8] Ying Dong, Wenbo Guo, Yueqi Chen, Xinyu Xing, Yuqing Zhang, and Gang Wang. Towards the detection of inconsistencies in public security vulnerability reports. In *Proc. 28th USENIX Security Symposium*, pp. 869–885, 2019.

[9] Afsah Anwar, Ahmed Abusnaina, Songqing Chen, Frank Li, and David Mohaisen. Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. *IEEE Transactions on Dependable and Secure Computing*, pp. 4255–4269, 2022.

[10] Viet Hung Nguyen and Fabio Massacci. The (un)reliability of NVD vulnerable versions data: an empirical experiment on google chrome vulnerabilities. In *Proc. 8th ACM SIGSAC Symposium on Information, Computer and Communications Security (ASIACCS '13)*, pp. 493–498, 2013.

[11] NVD JSON 2.0 Data Feeds. https://nvd.nist.gov/vuln/data-feeds.

[12] Fortinet PSIRT Advisories (CVRF, CSAF). https://fortiguard.fortinet.com/psirt.

[13] Cisco openVuln API. https://developer.cisco.com/docs/psirt/.

[14] Fortinet CVRF: supplement detection for historical advisories. https://github.com/MaineK00n/vuls-data-update/pull/892, https://github.com/MaineK00n/vuls-data-update/pull/941.

[15] Apple Security Releases fetcher and extractor. https://github.com/MaineK00n/vuls-data-update/pull/914, https://github.com/MaineK00n/vuls-data-update/pull/917.

[16] Microsoft servicing article fetcher. https://github.com/MaineK00n/vuls-data-update/pull/919.

[17] OpenSSH: Security. https://www.openssh.com/security.html.

[18] OpenSSH security advisory data source. https://github.com/MaineK00n/vuls-data-update/pull/928.

[19] vuls-data-db: openssh-security: detect stale conversions. https://github.com/vulsio/vuls-data-db/pull/224.

[20] Outpacing NVD in CPE Assignment. https://www.vulncheck.com/blog/outpacing-nvd-cpe.

[21] OpenSSH 10.5 release notes. https://www.openssh.com/txt/release-10.5.